GUIDE 35 · OPERATE

Wire up Prometheus and Grafana

The latest version is at anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=35-wire-up-prometheus-and-grafana. This copy was made on 4 November 2026.

Time: 20 minutes. **You'll end up with:** QueryAPIGate's metrics collected by Prometheus, the bundled Grafana dashboard showing traffic, errors, latency and per-connection load, and a few alert rules worth starting with.

QueryAPIGate serves Prometheus metrics at `/metrics` - no key needed - and ships a Grafana dashboard built on them. The Console's **Metrics** screen shows the same numbers as a snapshot; Prometheus adds history, trends and alerting.

Everything below was run for real: Prometheus scraping a server, the dashboard imported into Grafana, and every one of its panel queries returning data.

Step 1: Look at the metrics

```shell
curl http://127.0.0.1:5000/metrics
```

```text
queryapigate_requests_total{method="GET",endpoint="api.run_named_query",status="200",key="app-alice"} 12
queryapigate_queries_total{connection="shop",dialect="sqlite",status="success",key="app-alice"} 12
queryapigate_active_queries 0
queryapigate_pool_idle_connections 2
queryapigate_rate_limit_rejections_total 3
...
```

The main ones, by what they answer:

QUESTION	METRIC
How much traffic, and how many errors?	`queryapigate_requests_total` (by method, endpoint, status, key)
How slow are requests?	`queryapigate_request_duration_seconds` (histogram, by endpoint)
How busy is each database?	`queryapigate_queries_total`, `queryapigate_query_duration_seconds`, `queryapigate_rows_returned_total` (by connection)
Is anything stuck?	`queryapigate_active_queries`
Are callers being throttled?	`queryapigate_rate_limit_rejections_total`
Is the cache working?	`queryapigate_cache_hits_total`, `queryapigate_cache_misses_total`
Is run history keeping up?	`queryapigate_history_runs_total{outcome="dropped"}`, `queryapigate_history_pending`

The full list: Observability.

Step 2: Scrape it

```yaml
# prometheus.yml
scrape_configs:
  - job_name: queryapigate
    static_configs:
      - targets: ["queryapigate:5000"]       # the container name in Compose, or host:port
```

Check **Status → Targets** in Prometheus: the target should be `up`. Verified - `http://127.0.0.1:5055/metrics` showed `up`, and `up{job="queryapigate"}` returned `1`.

Scrape **each process** separately. The numbers are per process, in memory, reset on restart - Prometheus handles resets in `rate()` and `increase()`. If you run `queryapigate mcp` or `queryapigate events`, add their ports (5001, 5002) as targets too: each serves its own `/metrics`. MCP traffic appears with `method="MCP"`.

Keep `/metrics` away from the internet: it's public on QueryAPIGate by design (so Prometheus needs no key), and it names your connections and keys. Restrict it at the proxy - see Put QueryAPIGate behind a reverse proxy - or scrape over a private network.

Step 3: Import the dashboard

In Grafana: add a Prometheus data source pointing at your Prometheus, then **Dashboards → New → Import**, upload `documentation/grafana-dashboard.json`, and choose that data source when asked. (The same import over Grafana's API, `POST /api/dashboards/import` with the `DS_PROMETHEUS` input set, is how it was verified.)

The ten panels, all checked against live data:

- **Active queries**, **Pool idle connections**, **Rate limit rejections (1h)**, **Error rate (5m)** - stat panels for a glance.
- **Request rate by status** and **Request latency (p50 / p95 / p99)** - traffic and how it feels to callers.
- **SQL query rate by connection**, **SQL query latency p95 by connection**, **Rows returned by connection** - which database is busy, slow or returning a lot.
- **Response serialization latency p95 by format** - when large XLSX or CSV pages cost real time to encode.

Step 4: Alert on what matters

A starting set of Prometheus alerting rules - `promtool check rules` accepts it:

```yaml
groups:
  - name: queryapigate
    rules:
      - alert: QueryAPIGateDown
        expr: up{job="queryapigate"} == 0
        for: 2m
        annotations:
          summary: "QueryAPIGate {{ $labels.instance }} is not answering /metrics"
      - alert: QueryAPIGateServerErrors
        expr: sum(rate(queryapigate_requests_total{status=~"5.."}[5m])) /
sum(rate(queryapigate_requests_total[5m])) > 0.05
        for: 10m
        annotations:
          summary: "More than 5% of QueryAPIGate requests are failing with 5xx"
      - alert: QueryAPIGateSlowQueries
        expr: histogram_quantile(0.95, sum(rate(queryapigate_query_duration_seconds_bucket[5m])) by (le,
connection)) > 5
        for: 15m
        annotations:
          summary: "p95 query time on {{ $labels.connection }} is over 5 seconds"
      - alert: QueryAPIGateHistoryDropped
        expr: increase(queryapigate_history_runs_total{outcome="dropped"}[15m]) > 0
        annotations:
          summary: "Run history is dropping runs - the store can't keep up"
```

Alert on 5xx, not 4xx: a `400` or `403` is usually a caller's mistake, and the error-rate panel counts both. Tune the thresholds to your own traffic after a week of data.

The Console's **Alerts** screen covers the other half - expiring keys, a connection that keeps failing, a query that's often slow - things that need a person to act, rather than paging anyone.

Good to know

- **Label cardinality:** request and query counters carry the key name - fine for tens or hundreds of keys. Latency histograms deliberately don't, and all signed-in (JWT) users share one `jwt` label, so the series count stays bounded.
- **One worker per process** keeps the numbers whole; see Scaling out.

Next steps

- Read structured logs and trace one request end to end - from a spike on the dashboard to the request behind it.
- Diagnose and tune the connection pool - what the pool panel is telling you.